FECAP – Fundação Escola de Comércio Álvares Penteado
Análise e Desenvolvimento de Sistemas

KFKA

Estrutura de Dados

Grupo KFKA Tech

Vitor de Moura Lima
João Henrique Percino Albuquerque
Gabriel Barbosa da Silva Cabral

1. Visão geral

O KFKA é um sistema de acompanhamento escolar para o Ensino Fundamental. O professor lança o acompanhamento bimestral de cada aluno, a coordenação revisa e publica, e o responsável consulta os relatórios. O front-end foi feito em React e busca os dados em uma API em Node.js, que usa MySQL.

Como os dados chegam do back-end em forma de listas, boa parte do trabalho do front-end é paginar, agrupar, filtrar e exportar essas listas. Neste relatório mostramos quais estruturas de dados usamos para isso e por que elas deixam o sistema mais rápido.

Resumo das estruturas usadas

Estrutura	Onde usamos	Por que usamos
Lista com paginação	Hook **usePaginacao**, em todas as tabelas	A tela desenha só os itens da página atual
Tabela hash (Map)	Agrupamento dos indicadores em Relatórios	Os grupos são montados em uma única passada
Objeto chave-valor	**AGRUPAMENTOS** e **FILTROS_VAZIOS**	O rótulo e os filtros são acessados direto pela chave
Conjunto (Set)	Opções dos filtros de Relatórios	Os valores repetidos são descartados
Filtragem de lista	Filtros de Relatórios e busca das tabelas	Todos os critérios são testados de uma vez
Redução (reduce)	Função **calcularMedia**	A média sai sem criar listas intermediárias
Matriz	Função **exportarCsv**	Os arquivos do Excel são montados em linhas e colunas

2. Lista com paginação

> Paginar é dividir uma lista grande em partes menores e mostrar uma parte de cada vez.

Criamos o hook **usePaginacao** para não repetir a mesma lógica em cada tela. Ele recebe a lista e a quantidade de itens por página, calcula o total de páginas e usa o **slice** para pegar só os itens da página atual. O **useMemo** guarda esse resultado e só refaz o cálculo quando a lista ou a página mudam.

```jsx
import { useEffect, useMemo, useState } from 'react';

export default function usePaginacao(lista, itensPorPagina = 8) {
  const [pagina, setPagina] = useState(1);
  const totalPaginas = Math.max(1, Math.ceil(lista.length / itensPorPagina));

  useEffect(() => {
    if (pagina > totalPaginas) setPagina(totalPaginas);
  }, [pagina, totalPaginas]);

  const itensPagina = useMemo(
    () => lista.slice((pagina - 1) * itensPorPagina, pagina * itensPorPagina),
    [lista, pagina, itensPorPagina]
  );

  return { pagina, setPagina, totalPaginas, itensPagina };
}

```

Figura 1 – usePaginacao.js, linhas 1 a 17

Esse hook é usado em todas as tabelas do sistema: cadastros, revisão, portal do professor e relatórios. Na **TabelaCrud**, a busca filtra as linhas antes, e só o resultado vai para a paginação. Mesmo com centenas de alunos cadastrados, a tela desenha no máximo 8 linhas por vez.

```jsx
  const filtradas = useMemo(() => {
    const termo = normalizarTexto(busca);
    if (!termo) return linhas;
    return linhas.filter(l => l.valores.some(v => normalizarTexto(v).includes(termo)));
  }, [linhas, busca]);

  const { pagina, setPagina, totalPaginas, itensPagina } = usePaginacao(filtradas, 8);
```

Figura 2 – TabelaCrud.jsx, linhas 10 a 16

3. Tabela hash

> Tabela hash guarda pares de chave e valor. Com a chave, o valor é encontrado direto, sem precisar percorrer a estrutura, com custo O(1).

Na tela de Relatórios, o administrador pode agrupar os indicadores por turma, disciplina, professor, bimestre, aluno ou tag. Para montar esses grupos usamos um **Map**. O sistema passa uma única vez pelos acompanhamentos e coloca cada um na lista da sua chave. No caso das tags, o mesmo registro entra no grupo de cada tag que ele possui. Depois, cada grupo vira uma linha com a quantidade de registros, a média e o número de publicados, ordenada do maior grupo para o menor.

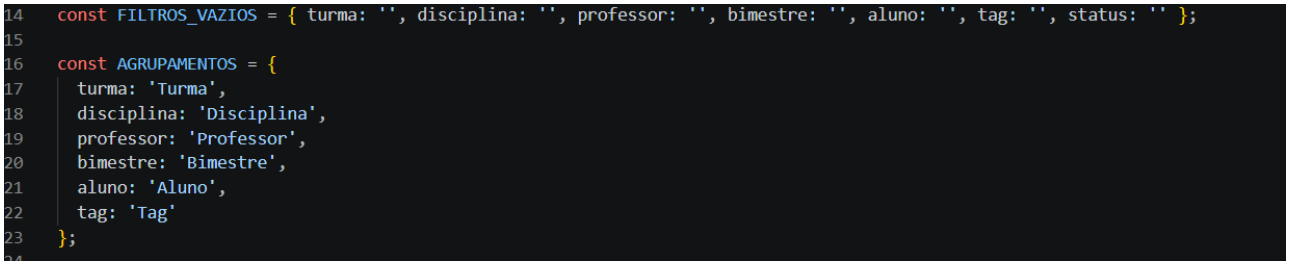
Sem o Map, teríamos que percorrer a lista inteira uma vez para cada grupo. Com ele, o agrupamento custa O(n), não importa quantos grupos existam.



```
91    const grupos = useMemo(() => {
92      const mapa = new Map();
93      filtrados.forEach(a => {
94        const chaves = agruparPor === 'tag' ? (a.tags?.length ? a.tags : ['(sem tag)']) : [a[agruparPor] || '—'];
95        chaves.forEach(chave => {
96          if (!mapa.has(chave)) mapa.set(chave, []);
97          mapa.get(chave).push(a);
98        });
99      });
100     const maiorQtd = Math.max(1, ...[...mapa.values()].map(l => l.length));
101     return [...mapa.entries()]
102       .map(([chave, lista]) => ({
103         chave,
104         qtd: lista.length,
105         media: calcularMedia(lista),
106         publicados: lista.filter(a => a.status === 'Publicado').length,
107         pct: Math.round((lista.length / maiorQtd) * 100)
108       }))
109       .sort((a, b) => b.qtd - a.qtd || String(a.chave).localeCompare(String(b.chave)));
110   }, [filtrados, agruparPor]);
```

Figura 3 – Relatorios.jsx, linhas 91 a 110

Os objetos **AGRUPAMENTOS** e **FILTROS_VAZIOS** funcionam do mesmo jeito. Em AGRUPAMENTOS, cada chave aponta para o seu rótulo, então **AGRUPAMENTOS[agruparPor]** já devolve o título da coluna e do arquivo exportado. FILTROS_VAZIOS tem um campo para cada filtro, e para limpar todos basta voltar para esse objeto.



```
14  const FILTROS_VAZIOS = { turma: '', disciplina: '', professor: '', bimestre: '', aluno: '', tag: '', status: '' };
15
16  const AGRUPAMENTOS = {
17    turma: 'Turma',
18    disciplina: 'Disciplina',
19    professor: 'Professor',
20    bimestre: 'Bimestre',
21    aluno: 'Aluno',
22    tag: 'Tag'
23  };
```

Figura 4 – Relatorios.jsx, linhas 14 a 23

4. Conjunto

> Conjunto (Set) é uma coleção que não aceita valores repetidos.

As opções dos filtros de Relatórios (turmas, disciplinas, professores, alunos e tags) são tiradas dos próprios acompanhamentos. Como o mesmo valor aparece em vários registros, passamos cada lista por um **Set**, que descarta as repetições, e depois ordenamos com **sort**. Nas tags usamos o **flatMap**, que junta as tags de todos os registros em uma lista só antes de passar pelo Set.



```
54    const opcoes = useMemo(() => {
55      const unicos = campo => [...new Set(acompanhamentos.map(a => a[campo]).filter(Boolean))].sort();
56      return {
57        turma: unicos('turma'),
58        disciplina: unicos('disciplina'),
59        professor: unicos('professor'),
60        aluno: unicos('aluno'),
61        tag: [...new Set(acompanhamentos.flatMap(a => a.tags || []))].sort()
62      };
63    }, [acompanhamentos]);
64
```

Figura 5 – Relatorios.jsx, linhas 54 a 63

5. Filtragem de listas

> Filtrar é percorrer uma lista e manter só os itens que atendem a uma condição.

Os sete filtros de Relatórios são verificados juntos, em uma única passada pela lista, ligados por **&&**. Quando um filtro está vazio, o teste **!filtros.campo ||** faz com que ele seja ignorado. Dessa forma, a lista não precisa ser percorrida uma vez para cada filtro.

```
65    const filtrados = useMemo(
66      () =>
67        acompanhamentos.filter(
68          a =>
69            (!filtros.turma || mesmoTexto(a.turma, filtros.turma)) &&
70            (!filtros.disciplina || mesmoTexto(a.disciplina, filtros.disciplina)) &&
71            (!filtros.professor || mesmoTexto(a.professor, filtros.professor)) &&
72            (!filtros.bimestre || a.bimestre === filtros.bimestre) &&
73            (!filtros.aluno || mesmoTexto(a.aluno, filtros.aluno)) &&
74            (!filtros.tag || (a.tags || []).includes(filtros.tag)) &&
75            (!filtros.status || a.status === filtros.status)
76        ),
77      [acompanhamentos, filtros]
78    );
```

Figura 6 – Relatorios.jsx, linhas 65 a 78

6. Redução e carregamento em paralelo

> Reduzir é percorrer uma lista acumulando um único resultado, como uma soma.

A função **calcularMedia** soma as médias com **reduce** e divide pelo tamanho da lista, sem criar listas intermediárias. Se a lista estiver vazia, ela devolve null em vez de dividir por zero. A mesma função calcula a média de cada grupo dos indicadores.

```
18  export const calcularMedia = lista =>
19    lista.length ? lista.reduce((acc, x) => acc + Number(x.media || 0), 0) / lista.length : null;
```

Figura 7 – formatadores.js, linhas 18 e 19

Ao abrir a tela de Relatórios, a lista de acompanhamentos e o histórico de auditoria são pedidos à API ao mesmo tempo, com **Promise.all**. Assim, a tela fica pronta no tempo da requisição mais lenta, e não na soma das duas.

```
36    const carregar = useCallback(async () => {
37      setCarregando(true);
38      setErro('');
39      try {
40        const [lista, hist] = await Promise.all([api.listar('acompanhamentos'), api.listarHistorico()]);
41        setAcompanhamentos(lista);
42        setHistorico(hist);
43      } catch (err) {
44        setErro(err.message);
45      } finally {
46        setCarregando(false);
47      }
48    }, []);
49
50    useEffect(() => {
51      carregar();
52    }, [carregar]);
```

Figura 8 – Relatorios.jsx, linhas 36 a 52

7. Matriz

> Matriz é uma lista de listas, organizada em linhas e colunas.

Para exportar os dados para o Excel, a função **exportarCsv** recebe o cabeçalho como uma lista e os dados como uma lista de linhas, ou seja, uma matriz. A função **celula** trata cada valor, tirando as quebras de linha e escapando as aspas, e o **join** junta as células com ponto e vírgula e as linhas com quebra de linha. Montar o texto dessa forma é mais eficiente do que ir concatenando dentro de um laço. A mesma função gera os três arquivos do sistema: acompanhamentos, indicadores por grupo e auditoria.



```
function celula(valor) {
  const texto = String(valor ?? '').replace(/[\r\n]+/g, ' ').replace(/"/g, '""');
  return `"${texto}"`;
}

export function exportarCsv(nomeArquivo, cabecalho, linhas) {
  const conteudo =
    '' + [cabecalho.map(celula).join(';'), ...linhas.map(l => l.map(celula).join(';'))].join('\r\n');

  const blob = new Blob([conteudo], { type: 'text/csv;charset=utf-8;' });
  const url = URL.createObjectURL(blob);
  const link = document.createElement('a');
  link.href = url;
  link.download = nomeArquivo;
  document.body.appendChild(link);
  link.click();
  link.remove();
  URL.revokeObjectURL(url);
}
```

Figura 9 – exportarCsv.js, linhas 1 a 19

Conclusão

Usamos estruturas de dados nas partes do KFKA que mais processam informação. A paginação limita o que vai para a tela, o Map agrupa os indicadores em uma passada, o Set monta as opções dos filtros sem repetição, os filtros combinados percorrem a lista uma vez só, o reduce calcula as médias e a matriz gera os arquivos do Excel. Com isso, as operações principais têm custo linear ou constante, e o sistema continua rápido mesmo com muitos registros.